

W Y K Ł A D 4

Funkcje w Pythonie

def, argumenty, zasięg, lambda, dekoratory

Wprowadzenie do programowania II

Informatyka Stosowana | semestr letni 2025/2026

Akademia Nauk Stosowanych w Nowym Sączu

Tematy

- 1 Definiowanie funkcji — `def`, `return`, `docstring`
- 2 Argumenty — pozycyjne, nazwane, `*args`, `**kwargs`
- 3 Separatory `/` i `*` — podział argumentów
- 4 Przestrzenie nazw i zasięg — reguła LEGB
- 5 Wyrażenia `lambda` — funkcje anonimowe
- 6 `map`, `filter`, `reduce` — programowanie funkcyjne
- 7 Domknięcia i dekoratory — closures, `@decorator`
- 8 Rekurencja, generatory i zaawansowane wzorce

Definiowanie funkcji

def, return, docstring, type hints

Anatomia funkcji w Pythonie

Funkcja to blok kodu wielokrotnego użytku, definiowany słowem kluczowym def.

Definicja z opisem każdego elementu

```
def pole_kola(promien):      # ← nagłówek
    """Oblicza pole koła.   # ← docstring

    Args:
        promien: długość promienia
    Returns:
        pole jako float
    """
    import math              # ← cięto
    return math.pi * promien ** 2 # ← return
```

Elementy składowe

def

Słowo kluczowe definiujące funkcję

nazwa

Identyfikator (snake_case)

parametry

Zmienne w nawiasach ()

docstring

Opis w potrójnych cudzysłowach

return

Zwraca wartość (lub None)

Instrukcja return

return kończy funkcję i zwraca wartość. Brak return → zwraca None.

Podstawowy return

```
def kwadrat(x):  
    return x ** 2  
  
print(kwadrat(5)) # 25
```

Zwracanie wielu wartości (krotka)

```
def min_max(dane):  
    return min(dane), max(dane)  
  
lo, hi = min_max([3,1,7,2]) # 1, 7
```

Brak return → None

```
def powitaj(imie):  
    print(f"Cześć, {imie}!")  
  
wynik = powitaj("Jan") # None
```

Wczesny return (guard clause)

```
def dziel(a, b):  
    if b == 0:  
        return None # wczesny wyjście  
    return a / b
```

Docstring i type hints

Dokumentacja i adnotacje typów nie zmieniają działania, ale poprawiają czytelność.

Docstring — help() i __doc__

```
def fib(n):  
    """Zwraca n-ty wyraz ciągu Fibonacciego."""  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a  
  
print(fib.__doc__) # Zwraca n-ty...
```

Type hints (PEP 484)

```
def powitaj(imie: str) -> str:  
    return f"Cześć, {imie}!"  
  
def srednia(nums: list[float]) -> float:  
    return sum(nums) / len(nums)  
  
# Python NIE wymusza typów!  
# To tylko podpowiedzi dla IDE
```



Ciekawostki

Funkcje są obiektami: można je przypisywać do zmiennych, wkładać do list, przekazywać jako argumenty.

Każda funkcja ma atrybuty: `__name__`, `__doc__`, `__defaults__`, `__code__`, `__annotations__`.

Argumenty funkcji

pozycyjne, nazwane, domyślne, *args, **kwargs

Argumenty pozycyjne i nazwane

Argumenty można przekazywać według pozycji lub nazwy (klucza).

Wywołanie pozycyjne

```
def info(imie, wiek, miasto):  
    print(f"{imie}, {wiek}, {miasto}")  
  
info("Anna", 25, "Kraków")  
# kolejność ma znaczenie!
```

Wywołanie z kluczem (nazwane)

```
info(wiek=25, miasto="Kraków",  
     imie="Anna")  
# kolejność dowolna  
# nazwane MUSZĄ być po pozycyjnych
```

Argumenty domyślne

```
def info(imie, wiek, miasto="Warszawa"): # miasto ma domyślną wartość  
    print(f"{imie}, {wiek}, {miasto}")  
  
info("Jan", 30) # Jan, 30, Warszawa ← użyto domyślnej  
info("Jan", 30, "Gdańsk") # Jan, 30, Gdańsk ← nadpisano
```


*args i **kwargs

Pozwalają przyjmować dowolną liczbę argumentów pozycyjnych lub nazwanych.

*args — krotka pozycyjnych

```
def suma(*args):  
    print(type(args)) # <class 'tuple'>  
    return sum(args)
```

```
suma(1, 2, 3)          # 6  
suma(10, 20, 30, 40)  # 100  
suma()                 # 0
```

**kwargs — słownik nazwanych

```
def config(**kwargs):  
    print(type(kwargs)) # <class 'dict'>  
    for k, v in kwargs.items():  
        print(f" {k} = {v}")
```

```
config(host="localhost",  
        port=8080,  
        debug=True)
```

Łączenie:

```
def fun(a, b, *args, **kwargs): # a, b — obowiązkowe, reszta opcjonalna
```

Rozpakowywanie argumentów

Operatory `*` i `**` działają też w drugą stronę — przy wywoływaniu funkcji.

`*` rozpakowuje sekwencję

```
def dodaj(a, b, c):  
    return a + b + c  
  
liczby = [1, 2, 3]  
dodaj(*liczby) # to samo co dodaj(1,2,3)  
  
# range → argumenty  
print(*range(5)) # 0 1 2 3 4
```

`**` rozpakowuje słownik

```
def powitaj(imie, wiek):  
    print(f"{imie} ma {wiek} lat")  
  
dane = {"imie": "Anna", "wiek": 25}  
powitaj(**dane)  
# to samo co powitaj(imie="Anna",  
#                       wiek=25)
```



Sprytne zastosowanie

```
defaults = {"host": "localhost", "port": 8080}  
custom = {"port": 3000, "debug": True}  
merged = {**defaults, **custom} # {'host': 'localhost', 'port': 3000, 'debug': True}
```

Separatory / i *

Od Pythona 3.8: / wymusza pozycyjność, * wymusza nazwy.

```
def f(pos1, pos2, /, pos_or_kwd, *, kwd1, kwd2):
    -----
    |           |           |           |
    |           | Positional or keyword |
    |           |           |           |
    |           |           | Keyword only
    |           |           |
    Positional only
```

Przykłady

```
def pos_only(a, b, /):  
    return a + b  
  
pos_only(1, 2)      # ✓ OK  
pos_only(a=1, b=2)  # ✗ TypeError!  
  
def kwd_only(*, a, b):  
    return a + b
```

Kolizja nazw — rozwiązanie z /

```
# BEZ / - kolizja!
def fun(nazwa, **kwsd):
    return "nazwa" in kwsd

fun(5, nazwa=10) # TypeError!

# Z / - OK!
def fun(nazwa, /, **kwsd):
    return "nazwa" in kwsd

fun(5, nazwa=10) # True ✓
```

Pułapka: mutowalny argument domyślny

Klasyczny błąd — domyślna lista/słownik jest współdzielona między wywołaniami!

X Źle

```
def dodaj(x, lista=[]):
    lista.append(x)
    return lista

print(dodaj(1)) # [1]
print(dodaj(2)) # [1, 2] !!!
print(dodaj(3)) # [1, 2, 3] !!!
```

✓ Dobrze

```
def dodaj(x, lista=None):
    if lista is None:
        lista = []
    lista.append(x)
    return lista

print(dodaj(1)) # [1]
print(dodaj(2)) # [2] ✓
```

Dlaczego?

Obiekt domyślny (lista, dict, set) jest tworzony RAZ — przy definicji funkcji, nie przy każdym wywołaniu.

Można to zobaczyć: `print(dodaj.__defaults__)` → pokazuje aktualny stan obiektu domyślnego.

Przestrzenie nazw i zasięg

Reguła LEGB, global, nonlocal

Reguła LEGB

Python szuka nazwy w czterech zasięgach — od największego do najszerzego.

L**Local**

zmienne wewnątrz bieżącej funkcji

E**Enclosing**

zmienne funkcji otaczającej (closure)

G**Global**

zmienne na poziomie modułu / pliku

B**Built-in**

print, len, int, list, range, type...

↑ Kierunek wyszukiwania: $L \rightarrow E \rightarrow G \rightarrow B$

LEGB w praktyce

Przykład zagnieżdżonych funkcji z różnymi zasięgami.

```
x = "GLOBAL"           # G

def outer():
    x = "ENCLOSING"     # E

    def inner():
        x = "LOCAL"     # L
        print(x)        # LOCAL

    inner()
    print(x)            # ENCLOSING

outer()
print(x)               # GLOBAL
```

Wyniki

inner()

LOCAL

outer() po inner

ENCLOSING

moduł główny

GLOBAL

Każda funkcja widzi swoje lokalne x,
które przesłania zmienne wyższych zasięgów.

global i nonlocal

Deklaracje pozwalające modyfikować zmienne spoza lokalnego zasięgu.

global — sięgamy do modułu

```
licznik = 0

def zwieksz():
    global licznik
    licznik += 1

zwieksz()
zwieksz()
print(licznik) # 2
```

nonlocal — sięgamy do enclosing

```
def zewnetrzna():
    x = 10
    def wewnetrzna():
        nonlocal x
        x += 5
    wewnetrzna()
    print(x) # 15

zewnetrzna()
```

⚠ global i nonlocal nie są zalecane do codziennego użytku!
Lepiej przekazywać dane przez argumenty i return — to bezpieczniejsze i łatwiejsze do testowania.

Mutowalne obiekty w funkcjach

Listy i słowniki przekazane do funkcji mogą być modyfikowane bez global/nonlocal.

Modyfikacja in-place

```
dane = [1, 2, 3]

def dodaj_element(lst):
    lst.append(4) # zmienia ORYGINAŁ!

dodaj_element(dane)
print(dane) # [1, 2, 3, 4]
```

Nowe przypisanie = nowy obiekt

```
dane = [1, 2, 3]

def reset(lst):
    lst = [0, 0, 0] # nowy obiekt!
    # NIE zmienia oryginału

reset(dane)
print(dane) # [1, 2, 3]
```

Zasada:

Python przekazuje REFERENCJĘ do obiektu. Metody in-place (append, update, extend) modyfikują oryginał. Operator = tworzy nową lokalną referencję — oryginał zostaje nietknięty. To samo dotyczy +=, -= dla int/str.

Wyrażenia lambda

Funkcje anonimowe i programowanie funkcyjne

Wyrażenia lambda

Jednolinijkowe funkcje anonimowe — bez nazwy, bez return, jedno wyrażenie.

Składnia

`lambda` parametry: wyrażenie

przykład:

```
kwadrat = lambda x: x ** 2
```

```
kwadrat(5) # 25
```

IIFE (natychmiastowe wywołanie)

```
(lambda x, y: x + y)(3, 4) # 7
```

więcej argumentów:

```
(lambda *a: sum(a))(1,2,3) # 6
```

Lambda vs def — porównanie

Cecha	lambda	def
Ilość wyrażen	Jedno	Wiele + instrukcje
Nazwa	Anonimowa (opcjonalna)	Zawsze nazwana
return	Niejawny (zawsze)	Jawny (opcjonalny)
Dekoratory @	Nie można użyć @	Można użyć @

Lambda w praktyce — sortowanie i klucze

Najczęstsze zastosowanie lambda: jako klucz sortowania.

```
# Sortowanie po długości słowa
words = ["Python", "is", "awesome", "!"]
sorted(words, key=lambda w: len(w)) # ['!', 'is', 'Python', 'awesome']

# Sortowanie listy krotek po drugim elemencie
students = [("Anna", 4.5), ("Jan", 3.8), ("Ewa", 5.0)]
sorted(students, key=lambda s: s[1], reverse=True)
# [('Ewa', 5.0), ('Anna', 4.5), ('Jan', 3.8)]

# Sortowanie słowników po wartości
scores = {"Alice": 95, "Bob": 87, "Charlie": 92}
sorted(scores.items(), key=lambda x: x[1], reverse=True)
# [('Alice', 95), ('Charlie', 92), ('Bob', 87)]

# max / min z kluczem
max(students, key=lambda s: s[1]) # ('Ewa', 5.0)
```

map, filter, reduce

Funkcje wyższego rzędu — przyjmują inną funkcję jako argument.

map(func, iter)

```
nums = [1, 2, 3, 4]
list(map(
    lambda x: x**2, nums
))
# [1, 4, 9, 16]
```

filter(func, iter)

```
nums = [1, 2, 3, 4, 5, 6]
list(filter(
    lambda x: x % 2 == 0, nums
))
# [2, 4, 6]
```

reduce(func, iter)

```
from functools import reduce
reduce(
    lambda a,b: a*b,
    [1,2,3,4,5]) # 120
```

map/filter vs comprehension

```
# Te zapisy są równoważne:
list(map(lambda x: x**2, nums))    ← map + lambda
[x**2 for x in nums]              ← list comprehension (bardziej pythonowe!)

list(filter(lambda x: x>3, nums))  ← filter + lambda
[x for x in nums if x > 3]         ← list comprehension (czytelniejsze!)
```

Domknięcia i dekoratory

Closures, @decorator, functools.wraps

Domknięcia (closures)

Funkcja wewnętrzna przechwytuje zmienne z otaczającego zasięgu.

```
def zewnetrzna(x):  
    y = 4  
    def wrap(z):  
        print(f"x={x}, y={y}, z={z}")  
        return x + y + z  
    return wrap      # zwracamy FUNKCJĘ
```

```
# Tworzenie domknięcia:  
closure = zewnetrzna(10)  
print(closure(5))  # x=10, y=4, z=5 → 19  
print(closure(8))  # x=10, y=4, z=8 → 22
```

```
# x i y są "zamknięte" w closure  
# choć zewnetrzna() już się zakończyła!
```

Jak to działa?

- 1 zewnetrzna(10) tworzy x=10, y=4
- 2 Definiuje wrap i zwraca ją
- 3 wrap "pamięta" x i y (enclosing)
- 4 closure(5) — z=5, reszta z pamięci
- 5 Można tworzyć wiele niezależnych domknięć

Pułapka: lambda w pętli

Lambda wiąże się z bieżącą wartością zmiennej — dopiero w momencie wywołania!

✗ Niespodzianka!

```
funkcje = []
for n in ["jeden", "dwa", "trzy"]:
    funkcje.append(lambda: print(n))

for f in funkcje:
    f()
# trzy
# trzy    <-- każda drukuje "trzy"!
# trzy
```

✓ Rozwiązanie: n=n

```
funkcje = []
for n in ["jeden", "dwa", "trzy"]:
    funkcje.append(lambda n=n: print(n))
    # n=n "zamraża" bieżącą wartość

for f in funkcje:
    f()
# jeden
# dwa
# trzy ✓
```

Wyjaśnienie:

Lambda NIE kopiuje wartości zmiennej w momencie definicji — trzyma referencję. Gdy pętla się kończy, n wskazuje na ostatnią wartość. Obejście: lambda n=n — argument domyślny jest ewaluowany OD RAZU.

Dekorator — podstawy

Dekorator to funkcja, która przyjmuje funkcję i zwraca jej ulepszoną wersję.

```
def dekorator(func):  
    def wrapper(*args, **kwargs):  
        print(" naglowek ".center(30, '-'))  
        wynik = func(*args, **kwargs)  
        print(" stopka ".center(30, '='))  
        return wynik  
    return wrapper  
  
@dekorator                # cukier składniowy!  
def dokument(tekst):  
    print(tekst)  
  
# Powyższe jest równoważne:  
# dokument = dekorator(dokument)
```

Przeptyw wywołania

```
dokument("Hello")  
  
↓ (= wrapper("Hello"))  
  
print(' naglowek '...)  
  
↓ func("Hello")  
  
print("Hello")  
  
↓ return  
  
print(' stopka '...)
```

functools.wraps — zachowanie tożsamości

Bez @wraps dekorowana funkcja traci nazwę i docstring. Zawsze używaj @wraps!

X Bez @wraps

```
def deco(func):  
    def wrapper(*a, **kw):  
        return func(*a, **kw)  
    return wrapper  
  
@deco  
def hello():  
    """Mówi hello."""  
    pass  
  
hello.__name__ # wrapper (!)  
hello.__doc__  # None (!)
```

✓ Z @wraps

```
import functools  
def deco(func):  
    @functools.wraps(func)  
    def wrapper(*args, **kwargs):  
        return func(*args, **kwargs)  
    return wrapper  
  
@deco  
def hello():  
    """Mówi hello."""  
    pass  
  
hello.__name__ # hello ✓  
hello.__doc__  # Mówi hello. ✓
```

⚠ Zawsze używaj @functools.wraps(func) w każdym dekoratorze — to standard!

Dekorator: pomiar czasu wykonania

Praktyczny przykład — dekorator mierzący czas działania funkcji.

```
import functools
import time

def timer(func):
    """Mierzy czas wykonania dekorowanej funkcji."""
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        wynik = func(*args, **kwargs)
        czas = time.perf_counter() - start
        print(f"{func.__name__}() zajęło {czas:.4f}s")
        return wynik
    return wrapper

@timer
def oblicz(n):
    return sum(i**2 for i in range(n))

oblicz(1_000_000) # oblicz() zajęło 0.0823s
```

Memoizacja — @lru_cache

Zapamiętywanie wyników — idealny dekorator do optymalizacji rekurencji.

Ręczna memoizacja (dekorator)

```
def pamiec(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@pamiec
def fib(n):
    return n if n < 2 else \
        fib(n-1) + fib(n-2)
```

Gotowe: functools.lru_cache

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fib(n):
    return n if n < 2 else \
        fib(n-1) + fib(n-2)

fib(100) # błyskawicznie!

# Statystyki cache:
print(fib.cache_info())
# CacheInfo(hits=98, misses=101,
#           maxsize=128, currsz=101)
```

Bez memoizacji fib(40) ≈ 100 sekund. Z memoizacją ≈ 0.0001 sekundy. Różnica: milion razy!

Rekurencja

Funkcja wywołująca samą siebie

Rekurencja — podstawy

Funkcja rekurencyjna wywołuje samą siebie. Każda potrzebuje warunku stopu!

Silnia — klasyczny przykład

```
def silnia(n):  
    if n <= 1:           # warunek stopu!  
        return 1  
    return n * silnia(n - 1)  
  
silnia(5) # 5*4*3*2*1 = 120
```

Fibonacci — rekurencja vs iteracja

```
# Rekurencyjnie ( $O(2^n)$  – wolne!)  
def fib(n):  
    if n < 2: return n  
    return fib(n-1) + fib(n-2)  
  
# Iteracyjnie ( $O(n)$ )  
def fib_iter(n):  
    a, b = 0, 1  
    for _ in range(n): a, b = b, a+b  
    return a
```

Limit rekurencji

```
import sys  
print(sys.getrecursionlimit()) # 1000 (domyślnie)  
sys.setrecursionlimit(5000)   # można zwiększyć, ale... ostrożnie z pamięcią stosu!
```

Rekurencja — sprytne przykłady

Rekurencja jest naturalna do przetwarzania struktur zagnieżdżonych.

Splaszczanie listy

```
def flatten(lst):  
    wynik = []  
    for el in lst:  
        if isinstance(el, list):  
            wynik.extend(flatten(el))  
        else:  
            wynik.append(el)  
    return wynik
```

Wieża Hanoi

```
def hanoi(n, src, dst, aux):  
    if n == 1:  
        print(f"{src} → {dst}")  
        return  
    hanoi(n-1, src, aux, dst)  
    print(f"{src} → {dst}")  
    hanoi(n-1, aux, dst, src)
```

Quick Sort — elegancka rekurencja

```
def quicksort(lst):  
    if len(lst) <= 1: return lst  
    pivot = lst[0]  
    mniejsze = [x for x in lst[1:] if x <= pivot]  
    wieksze = [x for x in lst[1:] if x > pivot]  
    return quicksort(mniejsze) + [pivot] + quicksort(wieksze)
```

Generator i yield

Leniwa ewaluacja i iteratory

Funkcje generatorowe — yield

yield "wstrzymuje" funkcję i zwraca wartość. Przy następnym wywołaniu kontynuuje.

Definicja generatora

```
def countdown(n):  
    while n > 0:  
        yield n      # "oddaje" wartość  
        n -= 1       # i kontynuuje stąd  
  
for x in countdown(5):  
    print(x)  # 5, 4, 3, 2, 1  
  
# Generator to iterator jednorazowy!
```

Generator vs lista — pamięć

```
# Lista: całość w pamięci  
kwadraty = [x**2 for x in range(10**7)]  
# ~80 MB pamięci!  
  
# Generator: Leniwy, O(1) pamięci  
kwadraty = (x**2 for x in range(10**7))  
# kilkadziesiąt bajtów!  
  
sum(kwadraty)  # działa normalnie
```

Ręczne pobieranie:

```
gen = countdown(3); next(gen)  # 3;  next(gen)  # 2;  next(gen)  # 1;  next(gen)  # StopIteration!
```

Generatory — praktyczne przykłady

Generatory są idealne do przetwarzania dużych danych strumieniowo.

Nieskończony ciąg

```
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

# Pierwsze 10 wyrazów:
fib = fibonacci()
for _ in range(10):
    print(next(fib), end=" ")
```

Przetwarzanie pliku linia po linii

```
def demo():
    a = 10
    b = "test"
    print(locals())
    # {'a': 10, 'b': 'test'}

demo()

# UWAGA: locals() zwraca KOPIĘ!
# Modyfikacja kopii nie zmienia zmiennych lokalnych.
```

Pipeline generatorów — łączenie w łańcuch

```
nums = range(1, 1_000_001)
kwadraty = (x**2 for x in nums) # gen 1
parzyste = (x for x in kwadraty if x%2==0) # gen 2
print(list(parzyste)[:5]) # pamięć O(1) przez cały pipeline!
```

Funkcje jako obiekty

First-class functions i wzorce

Funkcje pierwszej klasy

W Pythonie funkcje są obiektami — można je przypisywać, przekazywać, zwracać.

Przypisanie do zmiennej

```
def krzyk(tekst):  
    return tekst.upper() + "!!!"  
  
wolaj = krzyk      # BEZ nawiasów!  
wolaj("hej")      # "HEJ!!!"
```

Funkcja w liście / słowniku

```
ops = {  
    "+": lambda a,b: a+b,  
    "-": lambda a,b: a-b,  
}  
ops["+"](10, 3)  # 13
```

Funkcja jako argument i wartość zwracana

```
# Jako argument  
def zastosuj(func, dane):  
    return [func(x) for x in dane]  
zastosuj(str.upper, ["a", "b"])  # ['A', 'B']  
# Jako wartość zwracana (fabryka)  
def mnoznik(n): return lambda x: x*n  
mnoznik(2)(7)  # 14
```

Dekorator z argumentami

Potrzeba dodatkowej warstwy zagnieżdżenia — „fabryka dekoratorów”.

```
import functools

def powtorz(n):                # fabryka dekoratorów
    def dekorator(func):      # właściwy dekorator
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for _ in range(n):
                wynik = func(*args, **kwargs)
            return wynik
        return wrapper
    return dekorator

@powtorz(3)                    # powtorz(3) zwraca dekorator!
def przywitaj(imie):
    print(f"Cześć, {imie}!")

przywitaj("Anna")
# Cześć, Anna!
# Cześć, Anna!
# Cześć, Anna!
```

globals() i locals()

Python przechowuje przestrzeń nazw jako słowniki — można je podglądać!

globals() — zmienne globalne

```
x = 42
y = "hello"

print(globals()["x"]) # 42

# Dynamiczne tworzenie zmiennej!
globals()["z"] = 99
print(z) # 99

# globals() zwraca referencję
```

locals() — zmienne lokalne

```
def czytaj_logi(plik):
    with open(plik) as f:
        for linia in f:
            if "ERROR" in linia:
                yield linia.strip()

for err in czytaj_logi("app.log"):
    print(err) # plik 10 GB? OK!
```

⚠ **globals() i locals() to narzędzia do debugowania — nie do codziennego użytku.**

Pułapki i dobre praktyki

Typowe błędy i pythonowe idiomy

Częste błędy z funkcjami

Pięć najczęstszych pułapek, na które natrafiają początkujący.

Błąd	Poprawnie
print() zamiast return Funkcja zwraca None	return wynik (nie print!)
Mutowalny default: def f(x=[]): Lista współdzielona	def f(x=None):
Brak nawiasu: func vs func() Przekazuje obiekt, nie wywołuje	wynik = func() (z nawiasami)
Modyfikacja zmiennej globalnej Użyj return, nie global	return nowa_wartosc
Zbyt długa funkcja (100+ linii) 1 funkcja = 1 zadanie	Podziel na mniejsze funkcje

Dobre praktyki pisania funkcji

Zasady czytelnego, testowalnego kodu — PEP 8 i Python Zen.

✓	Nazwy w snake_case	oblicz_pole, pobierz_dane, wyslij_email
✓	Jedno zadanie na funkcję	Lepiej 5 małych niż 1 wielka
✓	Docstring dla publicznych funkcji	Co robi, jakie argumenty, co zwraca
✓	Return zamiast side-effects	Czyste funkcje są łatwiejsze do testowania
✓	Type hints dla złożonych parametrów	<code>def f(dane: list[int]) -> float:</code>
✗	Unikaj global i nonlocal	Przekazuj dane przez argumenty
✗	Unikaj mutowalnych domyślnych	Zawsze None + inicjalizacja w ciele
✗	Nie przekraczaj 3–4 argumentów	Więcej → rozważ słownik lub klasę

Callback i wzorzec strategii

Przekazywanie funkcji jako argumentu — elastyczny, rozszerzalny kod.

Callback — powiadomienie o zdarzeniu

```
def pobierz_dane(url, on_ok, on_err):  
    try:  
        dane = "..." # fetch  
        on_ok(dane)  
    except Exception as e:  
        on_err(str(e))  
  
pobierz_dane("https://api.ex.com",  
             on_ok=lambda d: print(f"OK: {d}"),  
             on_err=lambda e: print(f"ERR: {e}"))
```

Strategia — wymienna logika

```
def przetwarzaj(dane, strategia):  
    return [strategia(x) for x in dane]  
  
przetwarzaj([1,-2,3,-4], abs)  
# [1, 2, 3, 4]  
przetwarzaj(["abc","de"], len)  
# [3, 2]  
przetwarzaj([1,2,3], lambda x: x*10)  
# [10, 20, 30]
```



Gdzie to spotykamy?

sorted(key=...), map(func, ...), filter(func, ...), tkinter Button(command=...),
Django/Flask: dekoratory routów @app.route("/"), pytest fixtures, unittest mock

functools.partial i moduł operator

Przydatne narzędzia z biblioteki standardowej do programowania funkcyjnego.

partial — częściowe zastosowanie

```
from functools import partial
def potega(baza, wykladnik):
    return baza ** wykladnik

kwadrat = partial(potega, wykladnik=2)
szescian = partial(potega, wykladnik=3)
kwadrat(5)    # 25
szescian(3)   # 27
```

operator — operatory jako funkcje

```
import operator
from functools import reduce

reduce(operator.mul, [1,2,3,4,5]) #120

get_name = operator.itemgetter("name")
students = [{"name": "Jan"}, {"name": "Ala"}]
sorted(students, key=get_name)
```

Operator cheat sheet

operator.add	+	operator.mul	*	operator.eq	==
operator.sub	-	operator.truediv	/	operator.lt	<
operator.itemgetter(k)	d[k]	operator.attrgetter(a)	obj.a	operator.not_	not

Wbudowane dekoratory

Python dostarcza kilka gotowych dekoratorów — poznamy je dokładniej przy klasach.

Dekorator	Opis	Moduł
<code>@property</code>	Getter/setter jako atrybut	wbudowany
<code>@staticmethod</code>	Metoda bez self/cls	wbudowany
<code>@classmethod</code>	Metoda z cls zamiast self	wbudowany
<code>@abstractmethod</code>	Metoda abstrakcyjna (ABC)	abc
<code>@lru_cache</code>	Memoizacja wyników	functools
<code>@wraps</code>	Zachowuje <code>__name__</code> , <code>__doc__</code>	functools
<code>@dataclass</code>	Auto <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code>	dataclasses
<code>@total_ordering</code>	Generuje operatory porównania	functools

⚠ `@property`, `@staticmethod`, `@classmethod` omówimy szczegółowo na wykładzie o klasach.

Triki i ciekawostki

Sprytne jednoliniowce i mniej znane możliwości funkcji w Pythonie.

```
# 1. Wielokrotny dekorator (składanie)
@dekorator_A
@dekorator_B
def func(): ... # = dekorator_A(dekorator_B(func))

# 2. Fibonacci jednolinijkowy
fib = lambda n: n if n<2 else fib(n-1)+fib(n-2)

# 3. Debug dekorator jednolinijkowy
debug = lambda f: (lambda *a,**k: (print(f"-> {f.__name__}{a}"), f(*a,**k))[1])

# 4. Szybkie tworzenie wariantów
from functools import partial
bases = {b: partial(int, base=b) for b in [2,8,16]}
bases[2]("1010") # 10; bases[16]("ff") # 255

# 5. Podmiana print na debug
import builtins; old = builtins.print
def debug_print(*a,**k): old("[DBG]",*a,**k)
builtins.print = debug_print
```

Pod maską — moduł dis

Można podejrzeć bytecode Pythona. Lambda i def generują identyczny kod!

Lambda

```
>>> import dis
>>> prod = lambda a,b: a//b
>>> dis.dis(prod)
1  0 LOAD_FAST      0 (a)
  2 LOAD_FAST      1 (b)
  4 BINARY_FLOOR_DIVIDE
  6 RETURN_VALUE
```

Funkcja def

```
>>> def prod(a,b): return a//b
>>> dis.dis(prod)
1  0 LOAD_FAST      0 (a)
  2 LOAD_FAST      1 (b)
  4 BINARY_FLOOR_DIVIDE
  6 RETURN_VALUE

# Identyczny bytecode!
```

Wniosek:

Na poziomie bytecodu nie ma żadnej różnicy między lambda a def. Wybór to kwestia czytelności, nie wydajności. Lambda to cukier składniowy dla prostych, jednolinijkowych wyrażeń.

Imperatywnie vs funkcyjnie

Python pozwala na oba style — porównanie tego samego zadania.

Imperatywnie (pętla)

```
dane = [1, 2, 3, 4, 5, 6, 7, 8]

# Znajdź kwadraty parzystych
wynik = []
for x in dane:
    if x % 2 == 0:
        wynik.append(x ** 2)

print(wynik) # [4, 16, 36, 64]
```

Funkcyjnie (3 sposoby)

```
# 1. List comprehension (najlepszy!)
[x**2 for x in dane if x % 2 == 0]

# 2. map + filter
list(map(lambda x: x**2,
         filter(lambda x: x%2==0, dane)))

# 3. Pipelining z generatorami
parzyste = (x for x in dane if x%2==0)
kwadraty = (x**2 for x in parzyste)
list(kwadraty)
```

Rekomendacja: list comprehension jest najczęściej najczytelniejszą opcją.
Używaj map/filter gdy już masz gotową funkcję (np. str.upper, len).

Śledzenie obiektów — sys i weakref

Każdy obiekt w Pythonie ma licznik referencji. Można to podejrzeć!

sys.getrefcount()

```
import sys

a = [1, 2, 3]
print(sys.getrefcount(a)) # 2
# (+1 bo sam argument to referencja)

b = a
print(sys.getrefcount(a)) # 3

del b
print(sys.getrefcount(a)) # 2
```

weakref — słaba referencja

```
import weakref

class Foo: pass

a = Foo()
w = weakref.ref(a) # słaba ref.
print(w() is a)    # True

del a
print(w())          # None
# obiekt został usunięty!
```

weakref nie działa z: int, float, bool, str, bytes, tuple (prostymi, niemutowalnymi typami wbudowanymi).

Działa z: instancjami klas, listami, słownikami — obiektami mutowalnymi lub użytkownika.

Ściągą: składnia i wzorce

Kompaktowy przegląd najważniejszych wzorów z tego wykładu.

Składnia	Opis
<code>def f(a, b):</code>	Zwykła funkcja
<code>def f(a, b=10):</code>	Argument domyślny
<code>def f(*args):</code>	Zmienna liczba pozycyjnych (tuple)
<code>def f(**kwargs):</code>	Zmienna liczba nazwanych (dict)
<code>def f(a, /, b, *, c):</code>	Separatory: a pozycyjny, c nazwany
<code>lambda x: x**2</code>	Funkcja anonimowa
<code>f = partial(g, a=1)</code>	Częściowe zastosowanie
<code>@dekorator</code>	Dekoracja: <code>f = dekorator(f)</code>
<code>@lru_cache(128)</code>	Memoizacja wyników
<code>yield x</code>	Generator (leniwa ewaluacja)
<code>global x / nonlocal x</code>	Modyfikacja zmiennych spoza zasięgu
<code>f.__name__ / f.__doc__</code>	Introspekcja: nazwa i docstring

Fibonacci — ewolucja podejścia

Jedno zadanie, sześć różnych technik z tego wykładu.

```
# 1. Rekurencja prosta ( $O(2^n)$  – WOLNE!)
def fib1(n): return n if n<2 else fib1(n-1)+fib1(n-2)

# 2. Iteracyjnie ( $O(n)$ )
def fib2(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a+b
    return a

# 3. Memoizacja ręczna
@pamiec
def fib3(n): return n if n<2 else fib3(n-1)+fib3(n-2)

# 4. @lru_cache
@lru_cache
def fib4(n): return n if n<2 else fib4(n-1)+fib4(n-2)

# 5. Generator
def fib5():
    a, b = 0, 1
    while True: yield a; a, b = b, a+b

# 6. Lambda
fib6 = lambda n: n if n<2 else fib6(n-1)+fib6(n-2)
```

Funkcje w Pythonie

<code>def + return</code>	Definiują i zwracają — print to nie return!
<code>*args, **kwargs</code>	Elastyczne argumenty: krotka i słownik
<code>/ i *</code>	Separatory pozycyjnych i nazwanych
LEGB	Local → Enclosing → Global → Built-in
<code>lambda</code>	Jednolinijkowe funkcje anonimowe
<code>map, filter, reduce</code>	Programowanie funkcyjne (ale comprehension często lepsze!)
Closures	Funkcja pamięta enclosing scope
<code>@dekorator</code>	Funkcja opakowująca inną funkcję (+@wraps!)
<code>yield</code>	Generatory — leniwa ewaluacja, O(1) pamięci
First-class	Funkcje to obiekty: przypisuj, przekazuj, zwracaj

Następny wykład: **Moduły, pakiety i obsługa wyjątków**